Git e GitHub

Fundamentação teórica alinhada aos slides

Material de apoio ao instrutor para consulta no tablet durante o minicurso.

ENCOMP 2026
11 de novembro de 2026
Fedora ou Linux Mint, terminal Bash

Esta edição acompanha as nove páginas reais da apresentação e o percurso prático atual do minicurso.

Índice

Mapa rápido entre os slides e esta fundamentação 3

Introdução e abertura 3

1 Controle de versão 4

2 Git e GitHub 5

3 O repositório e os três estados do Git 6

4 Criação e configuração de um repositório 7

5 Commits e histórico 8

6 Branches 9

7 Merge e integração de históricos 10

8 Repositórios remotos e GitHub 11

9 Prática guiada: README, repositório e primeiro push 12

10 Erros comuns e formas de raciocinar 13

11 Compatibilidade com Fedora e Linux Mint 14

12 Síntese para o instrutor 15

Referências para continuar estudando 15

Material de apoio ao instrutor - ENCOMP 2026

Data do minicurso: 11/11/2026

Ambiente da prática: Fedora ou Linux Mint, utilizando o terminal Bash

Este material foi organizado para acompanhar a apresentação **Aula Git e Github ENCOMP**. O texto mantém a leitura contínua de uma fundamentação teórica, como um pequeno livro, mas agora indica em quais slides cada assunto aparece. Durante a aula, essas indicações funcionam como pontos de localização: você projeta o slide, consulta a explicação correspondente no tablet e desenvolve o assunto com suas próprias palavras.

Os números utilizados aqui representam as **nove páginas reais do PDF da apresentação**. Alguns slides ainda exibem números como 03/20, 06/20 ou 10/20 dentro do layout; essas marcações pertencem ao modelo visual e não são usadas para a navegação deste documento.

Mapa rápido entre os slides e esta fundamentação

Slide real	Conteúdo apresentado	Onde consultar neste material
1	Capa - Minicurso Git e GitHub	Introdução e abertura
2	Quem sou eu?	Apresentação do instrutor
3	Como saber o que mudou em um projeto?	Capítulo 1 - Controle de versão
4	Principais comandos	Capítulos 3, 4, 5 e 8
5	Termos essenciais	Capítulos 2, 5, 6, 7 e 8
6	Como ler um comando sem decorar?	Capítulo 4.3
7	Kahoot	Revisão antes da prática
8	Hora da prática	Capítulo 9 - Prática guiada no GitHub
9	Dúvidas e encerramento	Capítulos 10, 11 e 12

Introdução e abertura

Apoio ao slide 1 - Minicurso Git e GitHub

O minicurso começa apresentando uma ideia simples: Git e GitHub aparecem juntos com frequência, mas não são a mesma coisa. Antes de ensinar comandos, é importante mostrar qual problema essas ferramentas resolvem. A aula parte da dificuldade de acompanhar alterações em um projeto, apresenta o Git como ferramenta de controle de versão e, depois, mostra o GitHub como plataforma de hospedagem e colaboração.

Os alunos não precisam memorizar uma sequência inteira de comandos. O objetivo é entender o efeito de cada ação. Quando uma pessoa compreende que `git add` prepara uma alteração, que `git commit` registra uma versão local e que `git push` envia commits para o repositório remoto, os comandos deixam de parecer palavras isoladas.

Apoio ao slide 2 - Quem sou eu?

Este slide serve para uma apresentação breve. O ponto principal é explicar sua relação com o tema sem tentar assumir uma posição de especialista absoluto. Você é estudante de Ciência da Computação, possui experiência com projetos pessoais, automação, Python, Rust e web scraping e atua na área comercial da Adapti Soluções Web.

A Adapti ficou responsável por disponibilizar membros para o minicurso, e foi por meio dela que você se voluntariou para conduzir a aula. Essa informação ajuda a situar sua participação: você estudou, organizou o conteúdo e preparou uma experiência voltada a estudantes que também estão começando.

1 Controle de versão

Apoio ao slide 3 - Como saber o que mudou em um projeto?

1.1 O problema de trabalhar sem histórico

É comum começar um trabalho criando um arquivo e, depois de algumas alterações, salvar cópias com nomes como `trabalho-final`, `trabalho-final-v2` e `trabalho-final-agora-vai`. Essa estratégia parece funcionar enquanto há poucos arquivos e apenas uma pessoa trabalhando. O problema aparece quando precisamos descobrir qual cópia é a mais recente, recuperar uma parte apagada ou entender por que determinada mudança foi feita.

Copiar pastas também não explica o histórico. Sabemos que existem versões diferentes, mas não sabemos com clareza quem alterou cada arquivo, quando a mudança ocorreu ou qual era sua finalidade. Em um projeto de software, no qual muitos arquivos podem mudar ao mesmo tempo, esse método se torna difícil de controlar.

O controle de versão resolve esse problema ao registrar estados identificáveis do projeto. Cada registro preserva o conteúdo escolhido naquele momento e inclui informações que ajudam a compreender sua origem. Assim, podemos consultar versões anteriores, comparar mudanças e desenvolver novas ideias sem perder uma versão que já funciona.

No slide, as perguntas “Qual versão funciona?”, “O que mudou?”, “Por que mudou?” e “Como recuperar uma versão anterior?” devem ser tratadas como problemas reais. O Git não adivinha a intenção da equipe, mas fornece um histórico organizado para que essas perguntas possam ser respondidas.

1.2 O que um sistema de controle de versão registra

Um sistema de controle de versão mantém um histórico organizado. Ele não serve apenas para guardar cópias de segurança. Seu objetivo principal é explicar a evolução do projeto. Um bom histórico permite descobrir qual mudança introduziu um arquivo, qual era o estado anterior e quais alterações pertencem a uma funcionalidade.

Cada versão registrada deve representar uma mudança compreensível. Um projeto pode começar com a criação de uma página, receber uma correção no cadastro e, depois, ganhar uma nova funcionalidade. Se cada etapa estiver registrada separadamente, torna-se mais fácil revisar, testar e localizar problemas.

O histórico também facilita o trabalho em equipe. Cada pessoa pode desenvolver uma alteração em uma linha separada, registrar seu progresso e integrar o resultado quando estiver pronto. Mesmo quando uma única pessoa trabalha no projeto, essa separação ajuda a testar ideias com segurança.

1.3 Controle de versão e backup

O Git preserva o histórico dentro do repositório, mas um repositório armazenado somente em um computador ainda pode ser perdido se o disco falhar. Controle de versão e backup resolvem problemas relacionados, mas diferentes. O Git organiza a evolução dos arquivos. Uma cópia em outro dispositivo ou servidor ajuda a proteger o projeto contra a perda do equipamento.

Quando o projeto é enviado ao GitHub, passa a existir uma cópia remota do histórico enviado. Isso é útil, mas não transforma automaticamente o GitHub em uma estratégia completa de backup. Arquivos não commitados e commits que ainda não receberam `push` continuam apenas no computador local.

1.4 Versionamento local, centralizado e distribuído

Em um modelo local, as versões ficam organizadas no próprio computador. Em um modelo centralizado, um servidor concentra o histórico compartilhado. O Git utiliza um modelo distribuído: um clone completo normalmente possui arquivos e histórico suficientes para criar commits, comparar versões e trabalhar em branches sem consultar constantemente um servidor.

Uma equipe ainda pode escolher um repositório no GitHub como ponto principal de colaboração. A palavra “distribuído” não significa ausência de organização. Ela indica que o histórico não depende exclusivamente de uma única cópia central para as operações locais.

1.5 Uma breve história: Linux, Linus Torvalds e o Git

O Git foi criado em 2005 no contexto do desenvolvimento do kernel Linux, projeto liderado por Linus Torvalds. A comunidade precisava coordenar uma grande quantidade de contribuições feitas em paralelo. Velocidade, segurança do histórico, desenvolvimento não linear e funcionamento distribuído estavam entre as prioridades da nova ferramenta.

Essa história ajuda a compreender por que branches e operações locais ocupam uma posição tão importante no Git. Muitas pessoas precisavam trabalhar em partes diferentes do mesmo projeto e, depois, reunir essas contribuições.

Linus Torvalds está associado à criação do Git e do kernel Linux. Ele não criou o GitHub. O GitHub surgiu depois, em 2008, como uma plataforma que utiliza repositórios Git e acrescenta hospedagem, interface web e recursos de colaboração.

2 Git e GitHub

Apoio aos slides 4 e 5 - Comandos e termos essenciais

2.1 O que é Git

Git é um sistema de controle de versão distribuído. Ele é instalado no computador e pode funcionar sem conexão com a internet. Ao executar comandos como `git status`, `git add` e `git commit`, estamos conversando com o programa Git e modificando um repositório local.

O Git registra versões por meio de commits. Cada commit recebe um identificador e mantém relação com o commit anterior. Essa ligação forma o histórico. Quando existem branches e merges, o histórico deixa de ser uma linha simples e passa a formar um grafo, isto é, uma estrutura com caminhos que podem se separar e se reencontrar.

2.2 O que é GitHub

GitHub é uma plataforma de hospedagem e colaboração que trabalha com repositórios Git. Ela oferece interface web, armazenamento remoto, controle de acesso, acompanhamento de tarefas e revisão de contribuições por meio de pull requests.

GitHub e Git não são sinônimos. Git é a ferramenta que registra versões. GitHub é um serviço que hospeda repositórios e organiza a colaboração em torno deles. É possível usar Git sem GitHub. Também existem outras plataformas compatíveis com Git.

Uma comparação útil é pensar no Git como o mecanismo que organiza o histórico e no GitHub como um local compartilhado no qual esse histórico pode ser publicado e discutido. Os commits são criados pelo Git. O GitHub recebe esses commits quando alguém executa um envio para um repositório remoto.

2.3 Repositório local e repositório remoto

O repositório local fica no computador da pessoa. Ele contém os arquivos de trabalho e uma pasta oculta chamada `.git`, na qual o Git armazena o histórico, as branches e outras informações internas. Apagar essa pasta transforma o diretório em uma pasta comum e remove o histórico local.

Um repositório remoto é outra cópia acessível por um endereço. Esse endereço pode apontar para o GitHub ou para outro servidor. Por convenção, o primeiro remoto costuma receber o nome `origin`. Esse nome é apenas um apelido utilizado nos comandos.

Na prática atual, os alunos criarão um repositório remoto em suas próprias contas e conectarão o repositório local a ele. A autenticação será feita pelo VS Code, utilizando a entrada pelo navegador quando solicitada. A URL define para onde o Git enviará os commits; a autenticação confirma que a pessoa possui permissão para escrever naquele repositório.

3 O repositório e os três estados do Git

Apoio ao slide 4 - `status`, `add` e `commit`

3.1 Diretório de trabalho

O diretório de trabalho é a pasta visível na qual editamos os arquivos. Quando abrimos um projeto no VS Code, vemos o conteúdo desse diretório. Salvar um arquivo altera o conteúdo no disco, mas isso ainda não cria uma versão no Git.

Essa distinção é central para quem está começando. O editor salva arquivos; o Git registra versões. Um arquivo pode estar salvo e ainda aparecer como modificado. Isso significa que o conteúdo existe no disco, mas ainda não foi incluído em um novo commit.

3.2 Área de preparação

Antes de criar um commit, escolhemos quais mudanças farão parte dele. Essa seleção recebe o nome de área de preparação, ou *staging area*. O comando `git add` coloca nessa área o estado atual dos arquivos indicados.

A área de preparação permite separar mudanças. Uma pessoa pode preparar uma correção concluída e deixar outra mudança incompleta para depois. Executar `git add` não envia arquivos ao GitHub e não cria um commit. O comando apenas prepara conteúdo para o próximo registro local.

3.3 Repositório e histórico

Quando executamos `git commit`, o Git cria um novo registro com o conteúdo que estava preparado. Esse registro passa a fazer parte do histórico. Cada commit guarda uma identificação, informações de autoria, data, mensagem e ligação com o commit anterior.

O fluxo básico possui três etapas. Primeiro, a pessoa modifica e salva um arquivo no diretório de trabalho. Depois, `git add` coloca a versão escolhida na área de preparação. Por fim, `git commit` registra o conteúdo preparado no histórico.

O Git não move o arquivo entre três pastas físicas. Diretório de trabalho, área de preparação e histórico são estados lógicos utilizados para entender o que será registrado.

3.4 Como o Git descreve o estado

O comando `git status` é a principal fonte de orientação durante a aula. Ele mostra a branch atual e informa se existem arquivos novos, modificados ou preparados. Antes de repetir um comando ou tentar corrigir um erro, devemos consultar o estado.

Um arquivo novo que o Git ainda não acompanha aparece como *untracked*. Depois de `git add`, ele aparece como preparado. Um arquivo já registrado que foi alterado aparece como modificado. Depois que todas as mudanças acompanhadas são registradas, o Git informa que a árvore de trabalho está limpa.

Árvore limpa não significa que o projeto está correto ou publicado. Significa apenas que, naquele momento, não há diferenças pendentes entre os arquivos acompanhados e o último commit da branch atual.

4 Criação e configuração de um repositório

Apoio aos slides 4 e 6 - Principais comandos e leitura da sintaxe

4.1 Inicialização

O comando `git init` cria a estrutura interna de um repositório na pasta atual. No curso, usaremos `git init -b main` para criar o repositório já com a branch principal chamada `main`.

É importante observar a pasta atual antes da inicialização. Se o comando for executado na pasta errada, o repositório será criado no lugar incorreto. No Bash, `pwd` mostra o caminho atual e `ls` lista o conteúdo da pasta.

Executar `git init` várias vezes não corrige arquivos, commits ou branches. Quando algo parece errado, a primeira providência é verificar `pwd`, a branch atual e `git status`.

Versões antigas do Git podem não aceitar `git init -b main`. Nesse caso, é possível executar `git init` e, depois do primeiro commit, renomear a branch com `git branch -M main`. Essa alternativa deve ser usada apenas se a versão instalada realmente exigir.

4.2 Identidade dos commits

O Git registra nome e e-mail como parte da autoria de cada commit. Esses dados não são a senha do GitHub. Eles identificam quem criou o registro.

Em um computador pessoal, a configuração pode ser feita globalmente:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

Em computador compartilhado, é mais seguro utilizar `--local` dentro do repositório. A autenticação do GitHub acontece em outra etapa e concede permissão para o envio remoto.

4.3 Como ler um comando sem decorar a linha

Apoio direto ao slide 6

O slide separa o exemplo `git commit -m "mensagem"` em quatro partes:

- `git` é o programa;
- `commit` é a operação;
- `-m` é uma opção;
- `"mensagem"` é o valor recebido pela opção.

Um hífen costuma introduzir uma opção curta, como `-m`. Dois hífens costumam introduzir uma opção longa, como `--global` ou `--oneline`. A letra não possui um significado universal. Em `git commit -m`, a opção fornece a mensagem. Em `git branch -m`, a mesma letra está relacionada à renomeação. A opção deve ser lida junto do comando ao qual pertence.

As aspas fazem com que várias palavras sejam tratadas como um único argumento. Sem as aspas, cada palavra pode ser interpretada separadamente pelo terminal.

No slide 5 aparecem outras opções:

- `-c`, em `git switch -c`, cria uma branch e muda para ela;
- `-d`, em `git branch -d`, exclui uma branch que já foi integrada;
- `-u`, em `git push -u origin main`, registra a relação entre a branch local e a branch remota;
- `-m`, em `git commit -m`, recebe a mensagem do commit.

5 Commits e histórico

Apoio aos slides 4 e 5 - `commit` e registro de versões

5.1 O que é um commit

Um commit representa um estado registrado do projeto. Ele costuma ser descrito como uma fotografia, mas também conhece seu antecessor. Essa ligação permite percorrer o histórico e compreender como o projeto chegou ao estado atual.

O commit não precisa incluir todos os arquivos da pasta. Ele inclui o conteúdo que foi preparado. Por isso, é possível criar registros coerentes, cada um com uma finalidade.

5.2 Identificação e mensagem

Cada commit recebe um hash, exibido como uma sequência de letras e números. Os comandos geralmente mostram uma forma abreviada suficiente para identificar o commit dentro daquele repositório.

A mensagem explica a intenção da mudança. Mensagens como `atualização` ou `mudanças` dizem pouco. Uma mensagem como `Adiciona instruções ao README` permite compreender o objetivo sem abrir o conteúdo. Para iniciantes, uma frase curta com um verbo descrevendo a ação costuma funcionar bem.

5.3 Salvar, preparar e commitar são ações diferentes

Salvar no VS Code grava o conteúdo no arquivo. O Git percebe que o arquivo mudou, mas ainda não registra uma nova versão. `git add` escolhe o conteúdo. `git commit` cria o registro.

Essa diferença explica muitos erros iniciais. Se `git commit` disser que não há nada para registrar, talvez o arquivo não tenha sido salvo, não tenha sido preparado ou já esteja igual ao último commit. A análise começa com `git status`, e não com a repetição automática de todos os comandos.

5.4 Consulta e comparação

`git log` mostra os commits da branch atual, começando pelo mais recente. A opção `--oneline` resume cada commit em uma linha. Para observar branches e merges, podemos usar:

```
git log --oneline --graph --all --decorate
```

`--graph` desenha as linhas do histórico, `--all` inclui as branches conhecidas e `--decorate` mostra os nomes que apontam para cada commit.

`git diff` mostra mudanças que ainda não foram preparadas. Depois de `git add`, usamos `git diff --cached` para conferir o conteúdo que entrará no próximo commit. Um resultado vazio deve ser interpretado junto com `git status`.

6 Branches

Apoio ao slide 5 - Branch e `main`

6.1 O que é uma branch

Uma branch é um nome que aponta para um commit. Ela permite continuar o trabalho a partir de um ponto conhecido sem alterar imediatamente a branch principal. Quando novos commits são criados nessa branch, seu nome avança e passa a apontar para o commit mais recente daquela linha de desenvolvimento.

Branches são leves porque o Git não precisa copiar a pasta inteira para criá-las. O repositório continua sendo o mesmo. O que muda é a referência ativa e, quando necessário, o conteúdo apresentado no diretório de trabalho.

6.2 A branch principal

Neste curso, a branch principal se chama `main`. Ela costuma representar a linha principal do projeto. O nome é uma convenção. O Git não considera automaticamente que tudo o que está em `main` está correto; a equipe define como essa branch será utilizada.

6.3 Criar e trocar de branch

O comando abaixo cria uma branch e já muda para ela:

```
git switch -c nova-funcionalidade
```

Para voltar à branch principal:

```
git switch main
```

Trocar de branch pode alterar os arquivos visíveis porque cada branch pode representar um estado diferente do projeto. Um arquivo que desaparece ao trocar de branch não foi necessariamente apagado; ele pode existir apenas na outra linha de desenvolvimento.

7 Merge e integração de históricos

Apoio aos slides 4 e 5 - `merge`

7.1 O que é merge

Merge é a operação que integra uma linha de desenvolvimento em outra. Para integrar uma funcionalidade à `main`, primeiro mudamos para `main` e depois executamos o merge da branch desejada:

```
git switch main
git merge nova-funcionalidade
```

A branch que está ativa recebe a integração. `git merge nova-funcionalidade` não muda para a outra branch; ele traz para a branch atual o histórico alcançado por ela.

7.2 Fast-forward e commit de merge

Um merge pode acontecer por *fast-forward* quando a branch atual não possui commits novos e a outra branch está simplesmente adiante no mesmo caminho. Nesse caso, o Git move a referência da branch atual até o commit mais recente.

Quando duas linhas possuem commits diferentes, o Git pode criar um commit de merge com dois pais. Esse commit registra que os históricos foram reunidos. Para uma turma iniciante, o mais importante é compreender que branches separam trabalhos e merge reúne históricos.

7.3 Conflitos

Um conflito ocorre quando o Git não consegue decidir sozinho como combinar mudanças. Um caso comum aparece quando duas branches alteram de maneiras diferentes a mesma parte de um arquivo.

Conflito não significa que o projeto foi destruído. Significa que existem alternativas incompatíveis e alguém precisa escolher o conteúdo final. O procedimento geral consiste em ler os trechos marcados, editar o arquivo, preparar a resolução com `git add` e concluir o commit.

O conflito será explicado de forma conceitual. Ele não precisa ser provocado durante a prática principal.

8 Repositórios remotos e GitHub

Apoio aos slides 4 e 5 - Clone, origin, pull request, pull e push

8.1 Clone

`git clone` cria uma cópia local a partir de um repositório existente. Além dos arquivos, o clone recebe o histórico e configura o endereço de origem como um remoto chamado `origin`.

Um repositório público pode ser clonado por HTTPS sem permissão de escrita. Isso não dá ao aluno o direito de enviar alterações. Ler um repositório e escrever nele são ações diferentes.

8.2 Origin e conexão com o remoto

Quando um repositório já foi criado localmente, podemos adicionar o endereço remoto:

```
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git remote -v
```

`origin` é o nome convencional do remoto. A URL identifica o repositório. `git remote -v` permite conferir os endereços configurados.

8.3 Push

`git push` envia commits e referências locais para o repositório remoto. O commit precisa existir antes do push. Portanto, `push` não substitui `git add` nem `git commit`.

Na primeira publicação da `main`, utilizaremos:

```
git push -u origin main
```

A opção `-u` configura o acompanhamento entre a branch local `main` e a branch remota correspondente. Depois disso, os próximos envios normalmente podem ser feitos apenas com `git push`.

Estar conectado ao site do GitHub não garante, sozinho, que o terminal esteja autenticado. Na aula, o VS Code abrirá o fluxo de entrada pelo navegador. Depois da autorização, o Git poderá utilizar a credencial armazenada para enviar os commits.

8.4 Fetch e pull

`git fetch` consulta o remoto e atualiza as informações remotas conhecidas pela cópia local, sem integrar automaticamente o conteúdo à branch atual. `git pull` combina a obtenção dessas informações com uma integração.

Para uma primeira aula, basta compreender que `pull` recebe e integra atualizações, enquanto `push` envia commits. Em projetos reais, é necessário verificar a branch e o estado local antes dessas operações.

8.5 Pull request, fork, issue e GitHub Actions

Pull request é um recurso do GitHub para propor a integração de uma branch. Ele reúne descrição, comparação de arquivos, comentários e decisões de revisão. Pull request não é o mesmo que o comando `git pull`.

Fork é uma cópia de um repositório criada dentro de outra conta do GitHub. Ele é comum quando alguém deseja propor alterações a um projeto no qual não possui permissão direta de escrita.

Issue é um registro utilizado para relatar problema, sugerir melhoria ou organizar uma tarefa. GitHub Actions permite automatizar rotinas, como testes e publicação. Esses assuntos serão apresentados de forma breve para mostrar o que existe além do fluxo básico.

9 Prática guiada: README, repositório e primeiro push

Apoio ao slide 8 - Hora da prática

9.1 Objetivo

A prática liga os conceitos apresentados durante a aula. Cada aluno criará ou copiará um `README.md`, iniciará um repositório Git local, registrará o primeiro commit, criará um repositório vazio no GitHub e enviará o histórico.

O foco não está em desenvolver um sistema completo. O README funciona como um arquivo pequeno e fácil de conferir, permitindo observar todo o caminho entre edição, preparação, commit e publicação.

9.2 Preparar o projeto local

No terminal Bash, o aluno entra na pasta escolhida e confere o Git:

```
git --version
pwd
ls
```

Depois de criar ou copiar o `README.md`, inicializa o repositório:

```
git init -b main
git status
```

O `status` deverá mostrar o README como arquivo ainda não acompanhado.

9.3 Criar o primeiro commit

O arquivo é preparado e registrado:

```
git add README.md
git status
git commit -m "Adiciona README inicial"
git log --oneline
```

Depois de `git add`, o status deve indicar que o arquivo está pronto para o commit. Depois de `git commit`, o log deve mostrar o primeiro registro.

Se o Git pedir nome e e-mail, a identidade deve ser configurada e apenas o commit deve ser repetido. Não é necessário recriar o arquivo ou reiniciar o repositório.

9.4 Criar o repositório no GitHub

No GitHub, o aluno cria um repositório público. Como o README já existe localmente, o repositório remoto deve ser criado vazio, sem adicionar outro README, `.gitignore` ou licença nessa etapa. Isso evita começar com dois históricos diferentes.

Depois, o endereço HTTPS é copiado e configurado no terminal:

```
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git remote -v
```

O segundo comando serve para conferir se o endereço foi registrado corretamente.

9.5 Autenticar e enviar

O primeiro push será executado com:

```
git push -u origin main
```

Quando o VS Code solicitar autenticação, o aluno seguirá a entrada pelo navegador e autorizará o acesso. Depois do envio, a página do repositório deve ser atualizada no navegador. O `README.md` e o commit deverão aparecer.

Se o terminal informar que `origin` já existe, não adicione outro remoto. Use `git remote -v` para conferir o endereço. Se ele estiver incorreto, peça ajuda antes de alterá-lo.

9.6 Fazer uma segunda atualização

Para mostrar que o fluxo pode ser repetido, o aluno altera o README, salva e executa:

```
git status
git diff
git add README.md
git commit -m "Atualiza informações do README"
git push
```

O primeiro `push` configurou o destino com `-u`, por isso o segundo envio pode utilizar apenas `git push`. Ao atualizar o GitHub, o histórico deverá mostrar os dois commits.

10 Erros comuns e formas de raciocinar

Apoio ao slide 9 - Dúvidas e encerramento

10.1 Pasta errada

Quando o Git informa `not a git repository`, o problema costuma ser o diretório atual. `pwd` mostra onde estamos e `ls` ajuda a reconhecer os arquivos. A correção é entrar na pasta correta com `cd`. Não execute `sudo git` e não inicialize outro repositório antes de conferir o caminho.

10.2 Identidade ausente

Se o commit falhar por falta de nome ou e-mail, configure a identidade e repita somente o commit. Os arquivos preparados continuam na área de preparação. Consulte `git status` antes de repetir etapas.

10.3 Arquivo não preparado

Se o commit não inclui a mudança esperada, verifique se o arquivo foi salvo e preparado. `git status`, `git diff` e `git diff --cached` mostram onde está o conteúdo.

10.4 Remote origin already exists

Essa mensagem significa que o nome `origin` já foi utilizado. Confira o endereço:

```
git remote -v
```

Se o endereço estiver correto, continue sem repetir `git remote add`. Se estiver errado, o instrutor pode orientar a substituição com `git remote set-url origin URL_CORRETA`.

10.5 Push rejeitado

O push pode falhar por falta de autenticação, falta de permissão, endereço incorreto ou porque o repositório remoto já possui um histórico diferente. Leia a mensagem antes de aplicar qualquer correção.

Se o repositório remoto foi criado com um README pelo GitHub enquanto outro README já existia localmente, os históricos começaram separados. Para uma aula inicial, a solução mais clara costuma ser criar outro repositório remoto vazio, preservando a tentativa anterior para análise posterior.

10.6 Confusão entre Git e GitHub

Se um commit não aparece no GitHub, retome a separação: o commit foi criado localmente; somente o `push` envia o registro ao remoto. Da mesma forma, editar um arquivo diretamente no site não atualiza automaticamente a pasta local.

11 Compatibilidade com Fedora e Linux Mint

Os comandos Git utilizados no minicurso funcionam no Bash do Fedora e do Linux Mint. Comandos como `pwd`, `cd`, `mkdir`, `printf`, `cat` e `ls` também fazem parte do ambiente comum previsto para a aula.

Git e VS Code já deverão estar instalados. Se uma instalação excepcional for necessária antes do evento, Fedora utiliza `dnf`, enquanto Linux Mint utiliza ferramentas da família APT. Essa diferença pertence à preparação do laboratório e não deve consumir o tempo da aula.

Diferenças entre versões do Git podem afetar `git init -b main` e `git switch`. Em versões antigas, as alternativas são `git init` seguido de `git branch -M main`, e `git checkout` no lugar de `git switch`. O instrutor

deve começar verificando `git --version`.

Não utilize comandos exclusivos de PowerShell ou CMD na demonstração. Os caminhos projetados devem seguir a convenção Linux. Também não utilize `sudo git`, não desative o firewall, o SELinux ou a verificação TLS para resolver falhas de acesso.

12 Síntese para o instrutor

Apoio ao slide 9 - Revisão final e dúvidas

O encadeamento conceitual do minicurso é direto. O controle de versão resolve o problema de compreender a evolução de um projeto. Git registra essa evolução localmente por meio de commits. A área de preparação permite escolher o conteúdo do próximo commit. Branches criam linhas de desenvolvimento e merge integra essas linhas. GitHub hospeda repositórios e acrescenta recursos de colaboração.

Ao explicar qualquer comando, relacione-o a esse modelo:

- `git status` descreve o estado;
- `git add` prepara conteúdo;
- `git commit` registra uma versão local;
- `git log` apresenta o histórico;
- `git switch` muda a linha de desenvolvimento;
- `git merge` integra históricos;
- `git clone` copia um repositório;
- `git remote` administra endereços remotos;
- `git push` envia commits;
- `git pull` recebe e integra atualizações.

O Kahoot do slide 7 funciona como revisão antes da prática. Depois dele, o slide 8 marca a mudança entre explicação e execução. No encerramento, o slide 9 deve retomar principalmente quatro diferenças: salvar não é commitar; `git add` não cria commit; commit não é push; Git não é GitHub.

Quando o aluno entende o efeito de cada ação, deixa de depender da memorização de uma sequência mecânica. Esse é o principal resultado esperado do minicurso.

Referências para continuar estudando

- Documentação oficial do Git
- Livro Pro Git em português
- Documentação do GitHub em português
- Materiais e vídeos reunidos no repositório do minicurso

Este documento foi produzido no Windows. Os comandos foram escritos para Fedora e Linux Mint com Bash, mas não foram testados nos computadores da aula.